

TA Office Hours Schedule

SS 419 or TA semester reservations please schedule here or with Sarah in CS Office, SS 401, email: sarah1.colenso@umontana.edu, or phone: (406)243-28

Time	<u>Monday</u>	<u>Tuesday</u>	<u>Wednesday</u>	<u>Thursday</u>	<u>Friday</u>
9:00-9:50	Tara Brown CSCI 447		Tara Brown CSCI 447		Tara Brown CSCI 447 <i>ALCY</i>
10:00-10:50	Tim Van Driel CSCI 444	Tim Van Driel CSCI 444	Tim Van Driel CSCI 444		
11:00-11:50	Afra Rahman CSCI 332 <i>ALCY</i>		Afra Rahman CSCI 332		Afra Rahman CSCI 332
12:00-12:50					
1:00-1:50	Afra Rahman CSCI 332		Tim Van Driel CSCI 444 Tara Brown CSCI 447		
2:00-2:50	Jack Phillips CSCI 332	Jack Phillips CSCI 332	Jack Phillips CSCI 332 Jianru Shen CSCI 150		Jack Phillips CSCI 332
3:00-3:50	Bao Nguyen CSCI 232			Jianru Shen CSCI 150	Bao Nguyen CSCI 232
4:00-4:50	Bao Nguyen CSCI 232		Jianru Shen CSCI 150	Jianru Shen CSCI 150	Bao Nguyen CSCI 232

1. (6 points) Suppose you have algorithms with the following runtimes. (Assume these are exact running times as a function of the input size n , not asymptotic running times.) How much slower do these algorithms get when you double the input size? (You can find this by dividing the runtime on $2n$ by the runtime on n). You should simplify your answer as much as possible, including evaluating logarithms when possible.

(a) $5n^3$ $\frac{f(2n)}{f(n)} = \frac{5(2n)^3}{5n^3} = \frac{2^3 n^3}{n^3} = 2^3 = 8$

(b) $2n \log_2 n$ $\frac{f(2n)}{f(n)} = \frac{2(2n) \log_2(2n)}{2n \log_2 n} = \frac{2(\log_2 2 + \log_2 n)}{\log_2 n}$
 $= \frac{2(1 + \log_2 n)}{\log_2 n} = \frac{2}{\log_2 n} + \frac{2 \log_2 n}{\log_2 n} = \frac{2}{\log_2 n} + 2$

(c) 3^n

$$\frac{f(2n)}{f(n)} = \frac{3^{2n}}{3^n} = 3^{2n-n} = 3^n$$

2. (4 points) Simplify the following Big-O notation so that $f(n) = O(g(n))$. The function $g(n)$ should be both as simple and as tight as possible. For example, if $f(n) = 3n^2 + 5n + 2$, then the answer should be $O(n^2)$, even though $O(n^3)$ is also correct but not as tight, and $O(n^2 + 2)$ is also correct but not fully simplified.

(a) $f(n) = \log(6 \cdot n^{n+1})$. What is $g(n)$?

simplify $f(n) = \log(6 \cdot n^{n+1})$
 $= \log 6 + \log n^{n+1}$
 $= \log 6 + (n+1) \log n$
 $= \log 6 + n \log n + \log n$

if finished: can you generalize the doubling factors for polynomials, $(c \cdot n^a)$ and exponentials $(k \cdot b^n)$?

polynomials $c n^a$

constant doubling factor
 2^n
 okay

$$\frac{f(2n)}{f(n)} = \frac{\cancel{K} (2n)^a}{\cancel{K} n^a} = \frac{2^a \cancel{n^a}}{\cancel{n^a}} = 2^a$$

exponentials $K \cdot b^n$

non-constant dividing factor

$$\frac{\cancel{K} b^{(2n)}}{\cancel{K} b^n} = b^{2n-n} = b^n$$

logarithms $\log_2 n$ $\frac{1}{1}$

$$\begin{aligned} \frac{\log_2 2n}{\log_2 n} &= \frac{\log_2 2 + \log_2 n}{\log_2 n} \\ &= \frac{1}{\log_2 n} + \frac{\log_2 n}{\log_2 n} \\ &= \frac{1}{\log_2 n} + 1 \end{aligned}$$

Asymptotic Notation

$f(n), g(n)$

$f(n)$ is $O(g(n))$ if it is upper bounded by a constant multiple of $g(n)$ for sufficiently large n .

ex $f(n) = 10n^2 + 3n + 1000$

$$f(n) = O(n^2)$$

$$f(n) = O(n^3)$$

$f(n)$ is $\Omega(g(n))$ if it is lower bounded by a constant multiple of $g(n)$ for sufficiently large n .

is $f(n) = \Omega(n^2)$? T

$f(n) = \Omega(n^3)$? F

$f(n)$ is $\Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$